Week3_예습과제_명재서

4장 분류

4.1 분류(Classification)의 개요

- **정의**: 지도학습의 대표적인 유형으로, 기존 데이터의 피처와 레이블명(결정 값)을 학습하여 모델을 생성하고, 새로운 데이터가 주어졌을 때 그 레이블을 판별하는 것.

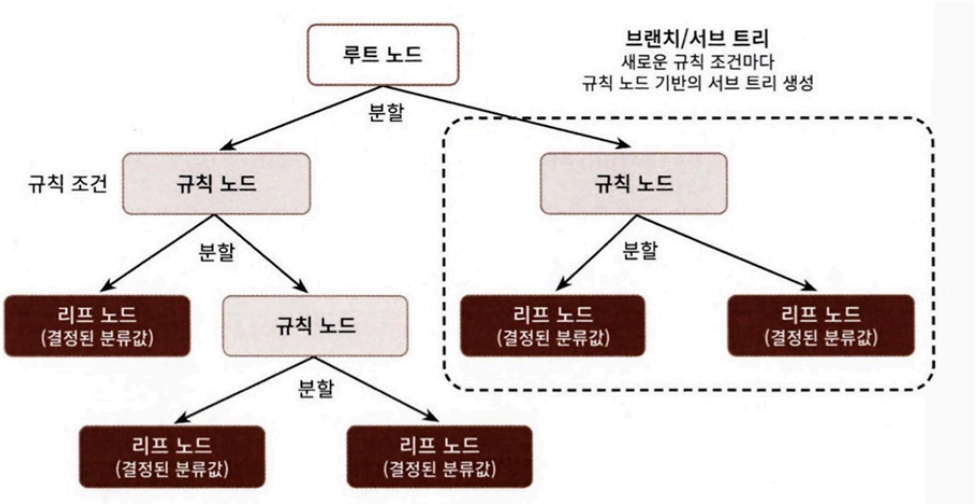
데이터의 피처 : 예측을 위한 힌트 / 입력값

레이블 : 맞춰야하는 정답 / 결과값

- **분류 알고리즘의 종류**:
  - 베이즈(Bayes) 통계와 생성 모델: 나이브 베이즈
  - 독립변수와 종속변수의 선형 관계: 로지스틱 회귀
  - 데이터 균일도에 따른 규칙 기반: 결정 트리 (Decision Tree)
  - 개별 클래스 간의 최대 마진: 서포트 벡터 머신 (SVM)
  - 근접 거리를 이용: 최소 근접 (Nearest Neighbor) 알고리즘
  - 여러 분류기를 결합: 앙상블 (Ensemble)
- **앙상블 학습의 특징**: 정형 데이터의 예측 분석 영역에서 매우 높은 예측 성능을 보임.
  - **배깅(Bagging)**: 랜덤 포레스트가 대표적이며 뛰어난 성능과 유연성을 가짐.
  - **부스팅(Boosting)**: GBM, XGBoost, LightGBM 등이 있으며 계속해서 발전 중.

  GridSearchCV

4.2 결정 트리 (Decision Tree)



- **개념**: 데이터에 있는 규칙을 학습을 통해 자동으로 찾아내 트리 기반의 분류 규칙을 만드는 것. (If-Then 규칙)
- **핵심 원칙 (정보의 균일도)**: 데이터를 분할할 때 최대한 균일한 데이터 세트가 구성되도록 분할하는 것이 중요함.
  - **정보 이득(Information Gain)**: 엔트로피 개념 기반. 엔트로피는 데이터의 혼잡도를 의미하며, 서로 다른 값이 섞여 있으면 높고 같은 값이면 낮음. `1 - 엔트로피 지수` 로 계산하며 정보 이득이 높은 피처를 기준으로 분할. - 정보 이득이 높은 속성을 기준으로 분할함.
  - **지니 계수(Gini Index)**: 0이 가장 평등(균일)하고 1로 갈수록 불평등(불순)함. 지니 계수가 낮은 피처를 선택해 분할. - 지니 계수가 낮은 속성을 기준으로 분할함
- **결정 트리 모델의 특징**:
  - **장점**: 균일도라는 룰을 기반으로 하고 있어 쉽고 직관적이며 피처 스케일링이나 정규화 같은 전처리가 필요 없음.
  - **단점**: 과적합(Overfitting)에 취약함. 트리가 깊어질수록 학습 데이터에만 과하게 최적화되어 실제 예측 성능이 떨어짐.
- **결정 트리 파라미터**:
  - `max_depth`: 트리의 최대 깊이.
  - `min_samples_split`: 노드를 분할하기 위한 최소 샘플 수.
  - `min_samples_leaf`: 말단 노드(Leaf)가 되기 위한 최소 샘플 수.
  - `max_features`: 최적의 분할을 위해 고려할 피처의 개수.
- **시각화 (Graphviz)**: `export_graphviz()` 를 사용하여 노드 내의 지니 계수, 샘플 수, 각 클래스별 데이터 수(value), 결정된 클래스(class)를 한눈에 확인 가능.

- **피처 선택 중요도**: `feature_importances_` 속성을 통해 알고리즘이 규칙을 만드는 데 중요하게 사용한 피처의 순위를 확인할 수 있음.

4.3 앙상블 학습 (Ensemble Learning)

- **정의**: 여러 개의 분류기를 생성하고 그 예측을 결합함으로써 보다 정확한 최종 예측을 도출하는 기법.
- **전략**: '약한 학습기'를 결합하여 '강한 학습기'를 만드는 것이 핵심.
- **유형**:
  1. **보팅(Voting)**: 서로 다른 알고리즘을 가진 분류기를 결합. (예: KNN + 로지스틱 회귀)
     - **하드 보팅**: 다수결 원칙.
     - **소프트 보팅**: 각 레이블별 확률의 평균을 구해 가장 높은 레이블 선택. (일반적으로 예측 성능이 더 좋아 주로 사용)
  2. **배깅(Bagging)**: 같은 알고리즘을 사용하되, 데이터를 샘플링하여 학습. (예: 랜덤 포레스트)
  3. **부스팅(Boosting)**: 여러 분류기가 순차적으로 학습하며, 앞선 분류기가 틀린 데이터에 대해 가중치를 부여해 다음 학습을 이어감.

4.4 랜덤 포레스트 (Random Forest)

- **원리**: 결정 트리를 기본 알고리즘으로 하며, 개별 트리가 학습하는 데이터는 전체 데이터에서 중복을 허용하여 샘플링한 **부트스트랩(Bootstrap)** 데이터 세트임.
- **장점**: 앙상블 알고리즘 중 비교적 빠른 수행 속도를 가지며 다양한 영역에서 높은 성능을 보임.
- **주요 파라미터**: `n_estimators`(결정 트리 개수), `max_features`(기본값은 'sqrt'와 동일).

4.5 GBM (Gradient Boosting Machine)

일반적으로 랜덤 포레스트보다 나은 예측 성능을 나타냄.

하지만 수행 시간이 오래 걸리고 하이퍼 파라미터 튜닝 노력이 더 필요.

- **작동 방식**: 가중치 업데이트를 경사 하강법(Gradient Descent)을 이용해 수행. 오류값(실제값 - 예측값)을 최소화하는 방향으로 반복해서 가중치를 업데이트함.

경사 하강법 : 반복 수행을 통해 오류를 최소화할 수 있도록 가중치의 업데이트 값을 도출하는 기법.

→ h(x) = y - F(x) [오류식 = 분류의 실제 결과값 - 피처에 기반한 예측 함수]

- **한계**: 순차적인 학습으로 인해 병렬 처리가 안 되어 수행 시간이 매우 오래 걸림.
- **주요 파라미터**:
- `loss` (경사 하강법에서 사용할 비용 함수),
- `learning_rate` (GBM이 학습을 진행할 때마다 적용하는 학습률, 약한 학습기가 순차적으로 오류 값을 보정해 나가는 데 적용하는 계수. 0~1 사이.). → n_estimator 상호보완적으로 조합해 사용함.
- `subsample` (약한 학습기가 학습에 사용하는 데이터의 샘플링 비율)

4.6 XGBoost (eXtreme Gradient Boosting)

- **특징**: GBM의 단점인 느린 속도와 과적합 규제 문제를 해결.
  - **표준화된 규제(Regularization)**: 과적합 방지.
  - **나무 가지치기(Tree Pruning)**: 긍정 이득이 없는 분할을 조기 중단.
  - **조기 중단(Early Stopping)**: 특정 횟수 동안 성능 개선이 없으면 학습 중단. → n_estimator에 지정한 부스팅 반복 횟수에 도달하지 않도라도 예측 오류가 더 이상 개선되지 않으면 반복을 끝까지 수행하지 않고 중단해 수행 시간을 개선 가능.
- **구현**: 파이썬 래퍼 XGBoost와 사이킷런 래퍼 XGBoost 두 가지 방식으로 사용 가능.
- **XGBoost의 주요 장점**

항목	설명
뛰어난 예측 성능	일반적으로 분류와 회귀 영역에서 뛰어난 예측 성능을 발휘합니다.
GBM 대비 빠른 수행 시간	일반적인 GBM은 순차적으로 Weak learner가 가중치를 증감하는 방법으로 학습하기 때문에 전반적으로 속도가 느립니다. 하지만 XGBoost는 병렬 수행 및 다양한 기능으로 GBM에 비해 빠른 수행 성능을 보장합니다. 아쉽게도 XGBoost가 일반적인 GBM에 비해 수행 시간이 빠르다는 것이지, 다른 머신러닝 알고리즘(예를 들어 랜덤 포레스트)에 비해서 빠르다는 의미는 아닙니다.
과적합 규제 (Regularization)	표준 GBM의 경우 과적합 규제 기능이 없으나 XGBoost는 자체에 과적합 규제 기능으로 과적합에 좀 더 강한 내구성을 가질 수 있습니다.
Tree pruning (나무 가지치기)	일반적으로 GBM은 분할 시 부정 손실이 발생하면 분할을 더 이상 수행하지 않지만, 이러한 방식도 자칫 지나치게 많은 분할을 발생할 수 있습니다. 다른 GBM과 마찬가지로 XGBoost도 max_depth 파라미터로 분할 깊이를 조정하기도 하지만, tree pruning으로 더 이상 긍정 이득이 없는 분할을 가지치기 해서 분할 수를 더 줄이는 추가적인 장점을 가지고 있습니다.
자체 내장된 교차 검증	XGBoost는 반복 수행 시마다 내부적으로 학습 데이터 세트와 평가 데이터 세트에 대한 교차 검증을 수행해 최적화된 반복 수행 횟수를 가질 수 있습니다. 지정된 반복 횟수가 아니라 교차 검증을 통해 평가 데이터 세트의 평가 값이 최적화 되면 반복을 중간에 멈출 수 있는 조기 중단 기능이 있습니다.
결손값 자체 처리	XGBoost는 결손값을 자체 처리할 수 있는 기능을 가지고 있습니다.

(250~257 : 실습)

4.7 LightGBM

- **특징**: XGBoost보다 학습 시간이 짧고 메모리 사용량도 적음.
- **리프 중심 트리 분할(Leaf-wise)**: 기존의 균형 분할(Level-wise)과 달리, 최대 손실값(max delta loss)을 가지는 리프 노드를 지속적으로 분할하여 깊고 비대칭적인 트리를 생성.
- **주의**: 적은 데이터 세트(10,000건 이하)에서는 과적합 발생 가능성이 높음.

4.10 스태킹 (Stacking)

- **원리**: 개별 모델이 예측한 데이터를 각각 쌓아서(Stack) 새로운 학습 데이터 세트를 만들고, 이를 **최종 메타 모델**이 다시 학습하여 예측하는 방식.
- **핵심**: 여러 모델의 강점을 결합하여 성능을 향상시키려 할 때 사용됨. (현업보다는 주로 캐글 등 대회에서 사용)

4.11 불균형한 데이터 세트 처리

- **문제점**: 레이블 분포가 극도로 불균형하면(예: 사기 탐지 0.1%), 모델은 무조건 다수 클래스로 예측하려는 경향이 생겨 성능 수치가 왜곡됨.
- **해결 방법**:
  1. **언더 샘플링(Under-sampling)**: 많은 레이블 데이터를 적은 레이블 수준으로 줄임.
  2. **오버 샘플링(Over-sampling)**: 적은 레이블 데이터를 증식시킴. 대표적으로 **SMOTE** 방식이 있으며, K-최근접 이웃을 이용해 데이터 사이의 가상 데이터를 생성함.